

SecretKeeper

Vault Structure

```
vault_folder/
├── name.txt
│   ├── pwd: <VALUE>
│   ├── otp: <VALUE>
│   └── anykey: <VALUE>
├── grandma.txt
│   ├── --- mail ---
│   │   ├── login: <VALUE>
│   │   ├── pwd: <VALUE>
│   │   └── phone: <VALUE>
│   └── --- AnOTher seCTion ---
│       ├── pwd: <VALUE>
│       └── anykey: <VALUE>
├── my_mail.txt
├── wifi_pwd.txt
├── encrypted.zip
│   ├── enc_pwd.txt
│   ├── wifi_pwd.txt
│   └── my_mail.txt
├── encrypted2.zip
│   └── wifi_pwd.txt
├── encrypted3.zip
│   └── some_pwd.txt
└── another_vault/
    ├── my_mail.txt
    ├── other_name.txt
    ├── encrypted3.zip
    └── some_pwd.txt
```

Get

```
sek get <name> [--key <key>] [--section <section>] [--action
<action>] [--pwd <password>]
sek get <name> [-k <key>] [-s <section>] [-a <action>] [-p
<password>]
```

– *name* – txt file name.

Resolution:

```
name          -> find unique match
/name         -> find in any vault root
/zipname/name -> find in zip in any vault
vault/name    -> specific vault root
vault/zip/name -> specific vault zip
```

```
vault/*/name    -> find unique match in specific vault
```

Options:

```
--key, -k          key (default: pwd)
--section, -s      section (default: *nameless*)
--action, -a       no | otp (default: no, otp for key=otp)
--pwd, -p          zip password; if omitted, master password is
requested
```

Examples:

```
# get pwd from vault_folder/name.txt
sek get name

# get other keys
sek get name -k otp
sek get name -k anykey

# get pwd from vault_folder/encrypted.zip/enc_pwd.txt
sek get enc_pwd

# get pwd from mail section of vault_folder/grandma.txt
sek get grandma -s mail

# get pwd from "    AnoTher  seCTion  " section of
vault_folder/grandma.txt
sek get grandma -s another_section

# same name in different zips
sek get wifi_pwd # ERROR: name is ambiguous

# get pwd from vault_folder/wifi_pwd.txt
sek get /wifi_pwd

# get pwd from encrypted.zip/wifi_pwd.txt
sek get /encrypted/wifi_pwd

# get pwd from encrypted2.zip/wifi_pwd.txt
sek get /encrypted2/wifi_pwd

# same name in different folders
sek get my_mail # ERROR: name is ambiguous

# get pwd from vault_folder/my_mail.txt
sek get vault_folder/my_mail

# get pwd from vault_folder/encrypted.zip/my_mail.txt
```

```
sek get vault_folder/encrypted/my_mail

# get pwd from another_vault/my_mail.txt
sek get another_vault/my_mail
```

Vaults

```
sek vault add <path to folder> [--primary] [--alias <alias>]
sek vault add <path to folder> [-p] [-a <alias>]
sek vault remove <path to folder>
sek vault list
```

alias	path	primary
vault_folder	.../vault_folder/	yes
another_vault	.../another vault/	no

Add

```
sek add <name> <value> [--key <key>] [--section <section>] [--
pwd <password>] [--overwrite]
sek add <name> <value> [-k <key>] [-s <section>] [-p <password>]
[-o]
```

Options:

```
--key, -k      key (default: pwd)
--section, -s  section (default: *nameless*)
--pwd, -p      zip password
--overwrite, -o allow overwrite
```

Zip password handling:

```
zip exists:
  if --pwd omitted, master password is requested

zip does not exist:
  master password is requested
  zip --pwd password is requested if not provided
```

Examples:

```
# add a new entry to the primary vault
sek add teapot 12345

# add a new entry to specific vault
sek add another_vault/teapot 12345

# add a new entry to the primary vault with a key and section
sek add kitchen 12345 -k otp -s teapot

# add a new entry to zip in the primary vault
```

```
# master password will be asked to save the zip file password

# zip does not exist, pwd is not specified
sek add /kitchen/teapot 12345
> enter master password:
> enter new zip password:
> enter new zip password again:

# zip does not exist, pwd is specified
sek add /kitchen/teapot 12345 --pwd qqg
> enter master password:

# zip already exist, pwd is not specified
sek add /kitchen/teapot 12345
> enter master password:

# zip already exist, pwd is specified
sek add /kitchen/teapot 12345 --pwd qqg

# add a new entry to zip in specific vault
sek add another_vault/kitchen/teapot 12345
```

Remove

```
sek remove <name> [--key <key>] [--section <section>] [--pwd
<password>]
sek remove <name> [-k <key>] [-s <section>] [-p <password>]
```

– Empty files are deleted.

Options:

```
--key, -k      key to remove (default: pwd)
--section, -s  section (default: *nameless*)
--pwd, -p      zip password; if omitted, master password is
requested
```

List

```
sek list [--all | -a]
```

Lists all entries.

--all requests the master password and includes encrypted entries.

name	full_name	is_encrypted	sections	keys
grandma	vault_folder/grandma	false	mail, another_section	mail:login, mail:pwd, mail:phone, another_section: pwd, another_section: anykey
name	vault_folder/name	false		login, pwd, phone
enc_pwd	vault_folder/encrypted/enc_pwd	true		pwd
/wifi_pwd	vault_folder/wifi_pwd	false		pwd
/encrypted/wifi_pwd	vault_folder/encrypted/wifi_pwd	true		pwd
vault_folder/my_mail	vault_folder/my_mail	false		pwd
/encrypted/my_mail	vault_folder/encrypted/my_mail	false		pwd
another_vault/my_mail	another_vault/my_mail	false		pwd
vault_folder/encrypted3/some_pwd	vault_folder/encrypted3/some_pwd	true		pwd
another_vault/encrypted3/some_pwd	another_vault/encrypted3/some_pwd	true		pwd

Master Password

```
sek set_master_pwd <old_master_password> <new_master_password>
```

Interactive:

```
sek set_master_pwd
> enter old master password:
> enter new master password:
> re-enter new master password:
```

Zip Passwords

Master Password is required to perform this actions.

```
sek zip_passwords set <zip_name> <password>
sek zip_passwords del <zip_name>
sek zip_passwords list [--missing_password | -m]
```

sek zip_passwords list

name	full_name	password is specified
encrypted	vault_folder/encrypted	yes
encrypted2	vault_folder/encrypted2	yes
vault_folder/encrypted3	vault_folder/encrypted3	yes
another_vault/encrypted3	another_vault/encrypted3	yes